

BEZPIECZEŃSTWO APLIKACJI W KONTENERACH

Damian Naprawa



WKONTENERACH.PL

Bezpieczeństwo aplikacji uruchamianych w kontenerach

Czyli jak poradzić sobie z podatnościami w obrazach

Od zaprezentowania świata Dockera minęło już kilka lat. W ciągu tego czasu stał się narzędziem, bez którego wiele osób nie wyobraża sobie codziennej pracy. Z drugiej strony – osobom, które dopiero zaczynają swoją przygodę z konteneryzacją może wydawać się, że Docker jest rozwiązaniem wszystkich ich dotychczasowych problemów – w tym również bezpieczeństwa aplikacji. Czy zatem aplikacja uruchomiona w kontenerze dockerowym jest bezpieczna?

W tym artykule zostanie poruszona bardzo ważna kwestia, jaką jest bezpieczeństwo obrazów. Całość przedstawiona będzie na przykładzie jednego z oficjalnych obrazów, którego ilość pobrań z Docker Huba w chwili pisania tego artykułu wynosi ponad **10 milionów**.

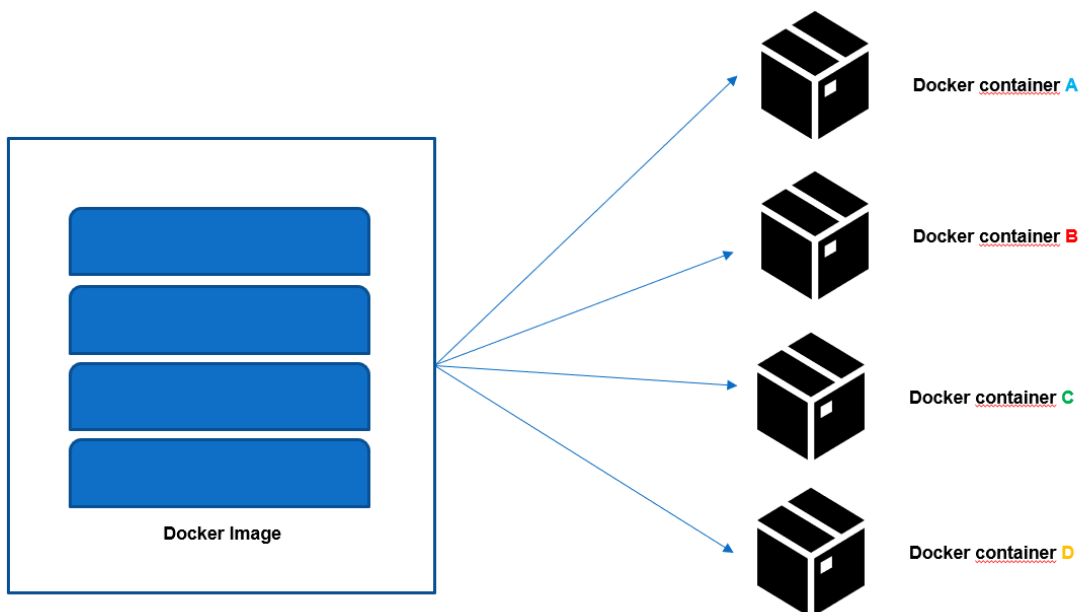
Aby lepiej zrozumieć, dlaczego należy zadbać samemu o bezpieczeństwo oraz skąd w obrazach mogą pojawić się podatności, na początku szczegółowo omówiona zostanie budowa obrazu. W dalszej części artykułu zostanie przedstawione główne źródło pochodzenia podatności w obrazach oraz narzędzia open-source, które można wykorzystać do ich wykrywania. Na koniec omówione zostaną dwie metody umożliwiające usunięcie wykrytych podatności z obrazów.

Czym jest obraz?

Docker image (inaczej obraz) jest swego rodzaju szablonem, na podstawie którego tworzony jest kontener. Od tego, jak zbudowany jest obraz, w dużej mierze zależeć będzie zachowanie kontenera po uruchomieniu. Co ważne, na podstawie jednego obrazu jesteśmy w stanie utworzyć wiele niezależnych od siebie instancji – kontenerów (patrz Rysunek 1).

Rysunek 1. Wiele kontenerów na podstawie obrazu





Do definiowania obrazu służy plik Dockerfile, na podstawie którego budowany jest obraz. Aby to lepiej zobrazować, weźmy za przykład Listing 0, w którym przedstawiono zawartość pliku Dockerfile, stworzonego dla aplikacji w języku NodeJS:

Listing 0. Przykładowy plik Dockerfile dla aplikacji NodeJS

```
FROM node:13.10

WORKDIR /usr/src/app

COPY package.json package-lock.json ./

RUN npm install

COPY /src ./src

CMD ["node", "./src/server.js"]
```

Za pomocą Instrukcji `FROM` określany jest obraz bazowy. W tym przypadku będzie to obraz `node:13.10`, który zawiera w sobie wszystkie niezbędne zależności potrzebne do uruchomienia aplikacji NodeJS. Więcej informacji na temat pochodzenia obrazów bazowych zostanie zaprezentowane w dalszej części artykułu.

Warto zwrócić uwagę, że za pomocą instrukcji `COPY` do obrazu dodajemy pliki `package.json` oraz `package-lock.json`, a następnie za pomocą instrukcji `RUN`



uruchamiane jest polecenie `npm install`. Ma to na celu przywrócenie zależności. Dopiero później kopiowane są pozostałe pliki. Kolejność instrukcji w pliku `Dockerfile` ma tutaj spory wpływ na finalną postać obrazu.

Znajdujące się w plikach `package*.json` zależności aplikacji zdecydowanie rzadziej zmieniają się aniżeli sam kod źródłowy. Dzięki temu podczas ponownego budowania obrazu zależności mogą zostać pobrane z pamięci cache. Wynika to warstwowej budowy obrazu, która zostanie bardziej szczegółowo omówiona w dalszej części artykułu. Obraz zbudowany na podstawie powyższej definicji posłuży do uruchomienia kontenera, gdzie głównym jego procesem będzie aplikacja NodeJS. Świadczy o tym polecenie określone w instrukcji `CMD`, czyli `node server.js`.

Do budowania obrazów służy polecenie `docker image build`. Aby obraz po zbudowaniu miał nazwę `mynodejs`, należy wykonać polecenie przedstawione w Listingu 1. Polecenie to należy wykonać z poziomu tego samego katalogu, w którym znajduje się plik `Dockerfile`. Wynika to z tego, że za pomocą znaku kropki ustawiany jest kontekst budowania obrazu jako bieżący katalog:

Listing 1. Budowanie obrazu

```
$ docker image build -t mynodejs .
```

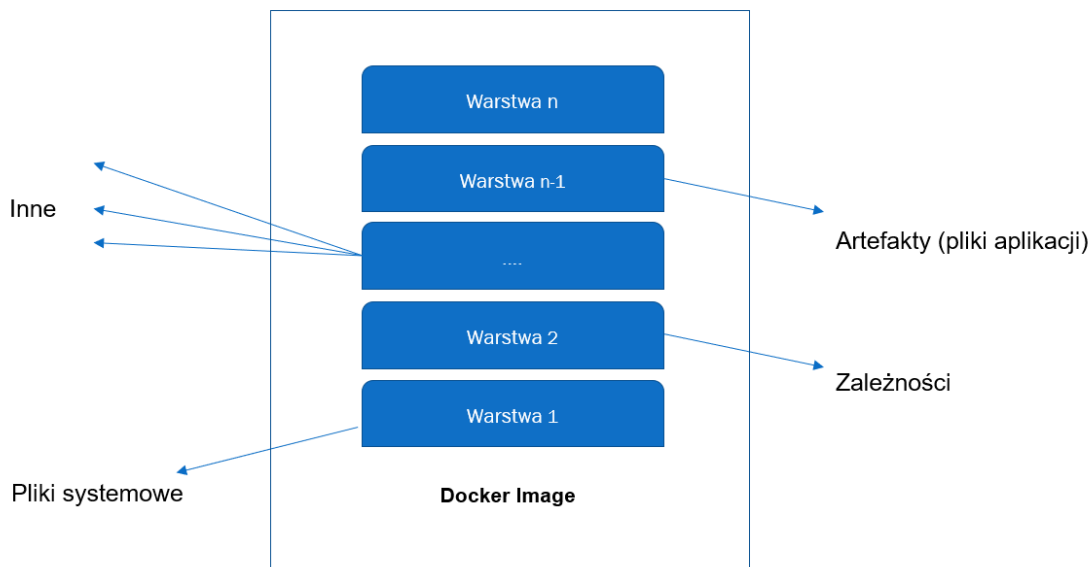
Z czego składa się obraz?

Jakby mogło się początkowo wydawać, obraz nie jest jednym plikiem, który fizycznie reprezentowałby jego strukturę. Każdy obraz składa się z wielu warstw oraz pliku konfiguracyjnego – tzw. pliku manifest.

Zwykle pierwsza warstwa zawiera specyficzne dla danej dystrybucji systemu operacyjnego pliki systemowe, które są niezbędne dla kolejnych warstw.

W większości przypadków głównym powodem, dla których tworzymy własne obrazy, jest potrzeba uruchomienia usług lub aplikacji. Każda z nich ma jakieś zależności. Może to być specyficzne dla danego języka programowania środowisko uruchomieniowe, lub po prostu zbiór bibliotek wykorzystywanych przez daną usługę czy aplikację. Za pomocą instrukcji `RUN` wymagane zależności są instalowane (zwykle przez menedżera pakietów, np. `apt`, `yum` – tym samym tworząc kolejną warstwę. Instrukcja `RUN` pozwala na uruchomienie dowolnego polecenia, które występuje w obrazie. Przykład: stworzenie katalogu za pomocą polecenia `mkdir`.

Rysunek 2. Przykład warstwowej budowy obrazu



Instrukcje `RUN`, `ADD` oraz `COPY` powodują utworzenie dodatkowych warstw. Pozostałe dostępne w Dockerfile instrukcje tworzą obrazy pośrednie, które nie zwiększają rozmiaru finalnego obrazu. W Listingu 2 przedstawiono przykład, gdzie dla każdej z instrukcji `RUN` utworzona zostanie osobna warstwa.

Listing 2. Instalowanie zależności w kilku krokach – niezalecany przykład

```
FROM debian:stretch

RUN apt-get update
RUN apt-get install -y ca-certificates
RUN apt-get install curl
```

Z kolei w Listingu 3 znajduje się przykład, gdzie po zbudowaniu obrazu zamiast trzech warstw zostanie dodana tylko jedna warstwa.

Listing 3. Instalowanie zależności w jednym kroku – dobry przykład

```
FROM debian:stretch

RUN apt-get update && apt-get install -y \
    ca-certificates \
    curl
```



Jeżeli zachodzi potrzeba zainstalowania kilku zależności – o ile to możliwe – należy zrobić to w jednym kroku (w jednej warstwie). Ma to na celu optymalizację miejsca na dysku oraz czasu pobierania obrazu. Wynika to z tego, że każda z warstw jest osobno pobierana, a następnie zapisywana na dysku. Jeżeli polecenie `apt-get update` zostawimy osobno, z każdym kolejnym budowaniem obrazu może zostać ono pominięte. Zamiast zaktualizowania listy pakietów w czasie budowania obrazu, lista pakietów zostanie pobrana z pamięci cache.

Zagłębiając się w szczegóły przechowywania warstw na dysku, należy wspomnieć, że każda z warstw zawiera również pliki i obiekty warstwy nadrzędnej. Na przykład warstwa druga zawierać będzie również pliki i obiekty warstwy pierwszej. Im więcej warstw, tym więcej miejsca na dysku będzie zajmować cały obraz. Wynika to z działania sterownika `overlay2`, który odpowiada za zarządzanie systemem plików obrazów i kontenerów. Więcej na ten temat można znaleźć w oficjalnej dokumentacji

<https://docs.docker.com/storage/storagedriver/select-storage-driver/>

Dlaczego warstwowa budowa obrazu?

Pierwszą zaletą jest przyspieszenie procesu budowania obrazów. Za całość odpowiada mechanizm cache'owania, który w trakcie budowania obrazów sprawdza, czy suma kontrolna (SHA256) dla plików poszczególnej warstwy zmieniła się. Jeżeli nie, proces budowania przechodzi dalej – do kolejnej warstwy. Dzięki takiemu rozwiązaniu proces budowania jest szybszy, gdyż budowane ponownie będą tylko te warstwy, dla których zmieniła się suma kontrolna.

Drugim powodem, dla którego warstwowa budowa obrazów ma sens, jest wielokrotne wykorzystywanie poszczególnych warstw dla różnych obrazów. Identyfikatorem warstwy jest suma kontrolna SHA256 z archiwum zawierającego pliki danej warstwy. Dzięki temu różne obrazy mogą wykorzystywać te same warstwy, pod warunkiem, że wszystkie warstwy nadrzędne dla dwóch różnych obrazów są również takie same.

Dziedziczenie warstw z obrazów bazowych

Dziedziczenie warstw z obrazów bazowych zostanie omówione na przykładzie oficjalnego obrazu NodeJS znajdującego się na Docker Hubie. Jest to oficjalne

repozytorium, gdzie publikowane są obrazy, z których każdy może skorzystać. Publikują je właściciele danego narzędzia/systemu. W przypadku projektów open-source obrazy są publikowane przez społeczność rozwijającą dany projekt.

Listing 4. Skrócony plik Dockerfile dla aplikacji NodeJS

```
FROM node:13.10

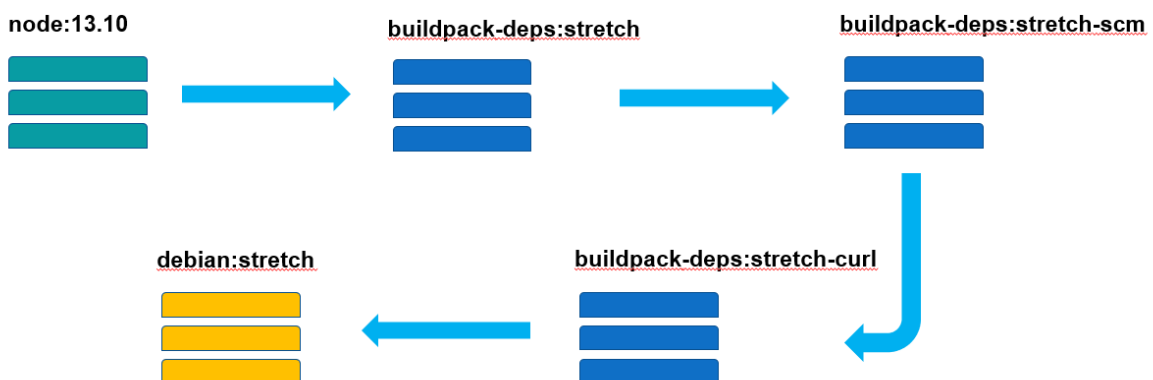
. . .

CMD ["node", "server.js"]
```

Instrukcja `FROM` w pliku Dockerfile powoduje, że obraz końcowy automatycznie dziedziczy wszystkie warstwy z obrazu bazowego. W tym konkretnym przypadku są to warstwy obrazu `node:13.10`. Dokładniej mówiąc, oprócz warstw utworzonych na podstawie zdefiniowanych przez użytkownika instrukcji w pliku Dockerfile, po zbudowaniu, finalny obraz będzie zawierał również wszystkie pliki i obiekty pochodzące z obrazu `node:13.10`.

Idąc dalej, należy zaznaczyć, że obraz `node:13.10` również został utworzony na podstawie obrazu bazowego, który z kolei korzysta z innego obrazu bazowego. Takich zagnieżdżeń może być znacznie więcej. Proces dziedziczenia został przedstawiony na Rysunku 3.

Rysunek 3. Dziedziczenie warstw obrazu końcowego opartego dla NodeJS



Na pierwszy rzut oka takie podejście może wydawać się nieco dziwne, ale jest to „standard” dla większości oficjalnych obrazów dostępnych na Docker Hubie. W powyżej przedstawionym przykładzie można zauważyć, że obraz `node:13.10` dziedziczy wszystkie warstwy – począwszy od obrazu `buildpack-deps:stretch`

– skończywszy na obrazie `debian:stretch`. To wszystko powoduje, że obraz końcowy również będzie mieć wszystkie warstwy pośrednie.

Jak dziedziczenie warstw wpływa na bezpieczeństwo obrazu końcowego?

Przykład obrazu NodeJS potwierdza, że proces dziedziczenia może być bardziej skomplikowany, niż mogło się to początkowo wydawać. Całość miała na celu ułatwić zrozumienie, dlaczego proces dziedziczenia warstw wpływa na bezpieczeństwo tworzonych obrazów. Aby jeszcze bardziej to zobrazować, w Listingu 5 przedstawiono plik Dockerfile dla obrazu `buildpack-deps:stretch-scm`, którego warstwy również znajdują się w finalnym obrazie `node:13.10`.

Listing 5. Dockerfile obrazu `buildpack-deps:stretch-scm`

```
FROM buildpack-deps:stretch-curl

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        bzip \
        git \
        mercurial \
        openssh-client \
        subversion \
        procps \
    && rm -rf /var/lib/apt/lists/*
```

Listing 5 potwierdza, dlaczego obrazy bazowe mogą wpływać na bezpieczeństwo obrazów końcowych. Za pomocą instrukcji `RUN` instalowane są różnego rodzaju narzędzia i aplikacje: `bzip`, `git`, `mercurial`, `openssh-client`, `subversion`, `procps`.

Wszystkie z nich będą znajdować się w końcowym obrazie aplikacji. Istnieje zatem ryzyko, że zainstalowane aplikacje i narzędzia mogą mieć podatności/luki bezpieczeństwa. Warto zaznaczyć, że jest to tylko jeden z wielu obrazów bazowych, z których dziedziczy obraz `node:13.10`. Pozostałe obrazy również mogą mieć podatności wynikające z zainstalowanych wewnątrz nich narzędzi.

Jak wykrywać podatności w obrazach?

Do wykrywania podatności w obrazach powstało wiele narzędzi, które są dostępne na rynku zarówno jako projekty open-source, jak i projekty komercyjne. W większej mierze uwaga zostanie poświęcona darmowym narzędziom.

Jednym z nich jest **Trivy**.

Działa na zasadzie analizy komponentów obrazu i sprawdza, czy występują w nich znane podatności, które mogą prowadzić do ataków lub naruszeń bezpieczeństwa.

Oto kilka cech, które czynią Trivy jednym z najlepszych narzędzi do skanowania obrazów Docker:

1. *Bogata baza danych podatności:* Trivy korzysta z różnych źródeł, takich jak bazy danych podatności i konfiguracji, aby zapewnić szerokie pokrycie podatności. Regularnie aktualizuje swoją bazę danych, aby uwzględniać najnowsze zagrożenia.
2. *Skanowanie wielu warstw obrazu:* Trivy analizuje każdą warstwę obrazu Docker, włącznie z zależnościami, aby zidentyfikować potencjalne podatności na różnych poziomach. To pozwala na dokładniejszą ocenę ryzyka.
3. *Wykrywanie podatności o różnym stopniu ryzyka:* Trivy ocenia podatności pod kątem stopnia ryzyka, oznaczając je jako niskie, średnie, wysokie lub krytyczne. Dzięki temu można skoncentrować się na najważniejszych zagrożeniach i podjąć odpowiednie działania.
4. *Wsparcie dla wielu języków i frameworków:* Trivy nie ogranicza się tylko do skanowania pod kątem podatności systemowych. Potrafi także identyfikować podatności związane z konkretnymi językami programowania, takimi jak Python, Ruby, Java czy JavaScript, oraz popularnymi frameworkami, na przykład Django czy Ruby on Rails.
5. *Prosta integracja:* Trivy jest łatwe w użyciu i może być łatwo zintegrowane z procesem CI/CD. Można go wykorzystać jako narzędzie wstępne, które automatycznie sprawdza obrazy Docker przed ich wdrożeniem, pomagając w zapewnieniu bezpiecznych i niezawodnych środowisk.
6. *Aktywna społeczność i rozwój:* Trivy jest projektem open-source, który posiada aktywną społeczność i regularne aktualizacje. Dodatkowo jest



ciągłe wspierane przez znaną w świecie IT Security firmę – Aqua Security Software. Dzięki temu narzędzie jest stale rozwijane, wprowadzane są nowe funkcje i naprawiane są ewentualne błędy.

Trivy jest uznawane za *jedno z najlepszych narzędzi do skanowania obrazów*, ponieważ oferuje szerokie pokrycie podatności, precyzyjną ocenę ryzyka i łatwą integrację w systemach CI/CD.

Listing 6. Skanowanie obrazów z Trivy

```
$ trivy image --severity CRITICAL, HIGH node:13.10
```

Po zakończeniu skanowania zwracana jest lista wykrytych podatności, wraz z informacją o stopniu wykrytej podatności (w tym przypadku tylko High i Critical) oraz linkami, w których zwykle znajduje się status podatności. Rezultat skanowania został przedstawiony na Rysunku 4.

Rysunek 4. Rezultat skanowania obrazu node:13.10 (tylko podatności CRITICAL)

node:13.10 (debian 9.12)
Total: 218 (CRITICAL: 218)

Library	Vulnerability	Severity	Installed Version	Fixed Version	Title	
bzip2	CVE-2019-12900	CRITICAL	1.0.6-8.1		bzip2: out-of-bounds write in function BZ2_decompress https://avd.aquasec.com/nvd/cve-2019-12900	
dpkg	CVE-2022-1664		1.18.25	1.18.26	Dpkg::Source::Archive in dpkg, the Debian package management system, b ... https://avd.aquasec.com/nvd/cve-2022-1664	
dpkg-dev						
imagemagick	CVE-2017-14532		8:6.9.7.4+dfsg-11+deb9u7	8:6.9.7.4+dfsg-11+deb9u10	ImageMagick: NULL pointer dereference in the TIFFIgnoreTags function https://avd.aquasec.com/nvd/cve-2017-14532	
	CVE-2017-14624				ImageMagick: NULL pointer dereference in the PostscriptDelegateMessage function https://avd.aquasec.com/nvd/cve-2017-14624	
	CVE-2017-14625				ImageMagick: NULL pointer dereference in the pixel_output_create function https://avd.aquasec.com/nvd/cve-2017-14625	
	CVE-2017-14626				ImageMagick: NULL pointer dereference in the pixel_decode function https://avd.aquasec.com/nvd/cve-2017-14626	
	CVE-2017-18211				ImageMagick: NULL pointer dereference in saveBinaryCLProgram in magick/opencl.c https://avd.aquasec.com/nvd/cve-2017-18211	
	CVE-2018-14551				8:6.9.7.4+dfsg-11+deb9u9	ImageMagick: Uninitialized variable in coders/mat.c:ReadMATImageV4() allows for memory corruption https://avd.aquasec.com/nvd/cve-2018-14551
	CVE-2019-19949				ImageMagick: heap-based buffer over-read in WritePNGImage in coders/png.c https://avd.aquasec.com/nvd/cve-2019-19949	
imagemagick-6-common	CVE-2017-14532	8:6.9.7.4+dfsg-11+deb9u10	8:6.9.7.4+dfsg-11+deb9u10	ImageMagick: NULL pointer dereference in the TIFFIgnoreTags function https://avd.aquasec.com/nvd/cve-2017-14532		
	CVE-2017-14624			ImageMagick: NULL pointer dereference in the PostscriptDelegateMessage function https://avd.aquasec.com/nvd/cve-2017-14624		
	CVE-2017-14625			ImageMagick: NULL pointer dereference in the pixel_output_create function https://avd.aquasec.com/nvd/cve-2017-14625		
	CVE-2017-14626			ImageMagick: NULL pointer dereference in the pixel_decode function https://avd.aquasec.com/nvd/cve-2017-14626		
	CVE-2017-18211			ImageMagick: NULL pointer dereference in saveBinaryCLProgram in magick/opencl.c https://avd.aquasec.com/nvd/cve-2017-18211		
	CVE-2018-14551			8:6.9.7.4+dfsg-11+deb9u9	ImageMagick: Uninitialized variable in coders/mat.c:ReadMATImageV4() allows for memory corruption https://avd.aquasec.com/nvd/cve-2018-14551	

Jak widzisz, Trivy wykryło jedyne 218 krytycznych podatności (na dzień, w którym wykonałem skanowanie). Pewnie zżera Cię teraz ciekawość, ile będzie podatności typu HIGH? Bardzo proszę, znajdziesz je na Listingu nr 7.



Listing 7. Łączna liczba podatności typu HIGH i CRITICAL dla obrazu node:13.10

```
node:13.10 (debian 9.12)
=====
Total: 1287 (HIGH: 1069, CRITICAL: 218)
```

Tym razem jedyne 1069 podatności typu HIGH i łącznie 1287 podatności, które powinny dać nam do myślenia. Z dziennikarskiego obowiązku przypomnę tylko, jest to obraz nadal (na czas pisania tego e-booka) dostępny na Docker Hub.

Oprócz manualnego skanowania, Trivy może zostać dodane jako krok w procesach ciągłej integracji/wdrażania (CI/CD). W łatwy sposób można to zrobić np. w Jenkinsie, Gitlabie czy Github Actions (ale jestem pewien, że sprawdzi się również w innych narzędziach CI/CD).

Inne narzędzia do skanowania

Szukając alternatywy dla Trivy, innymi i zarazem godnymi polecenia narzędziami do skanowania obrazów są [Grype](#) oraz [Clair](#). Grype wydaje się szczególnie godnym uwagi narzędziem, ponieważ oprócz bycia projektem open-source, jest rozwijane przez konkurencyjną firmę dla AquaSec. Mianowicie firmę Anchore, która dostarcza oprogramowanie zabezpieczające nie tylko obrazy dockerowe czy środowiska Kubernetes, lecz cały łańcuch dostaw oprogramowania (*Software Supply Chain Security*).

Jak usuwać podatności z obrazów?

Zanim jednak przejdziemy do samodzielnego usuwania podatności i budowania własnych obrazów, warto sprawdzić, czy na Docker Hubie nie pojawiła się nowsza wersja obrazu, niemająca w sobie tej podatności.

Od jakiegoś czasu dostawcy obrazów zwracają na bezpieczeństwo większą uwagę i sukcesywnie usuwają z nich podatności. Jeżeli jednak nie możemy czekać na oficjalny fix od dostawcy, możemy skorzystać z dwóch poniższych sposobów i usunąć podatności samodzielnie – tworząc tym samym własny obraz.

Usuwanie podatności – sposób pierwszy



Pierwszym sposobem na usunięcie podatności z obrazu jest utworzenie nowego pliku Dockerfile, w którym zostaną doinstalowane łatki dla wykrytych podatności. W Listingu 8 przedstawiono przykład dodatkowej instrukcji `RUN`, który jest odpowiedzialna za zainstalowanie nowszej wersji Pythona, która na dzisiaj jest wolna od podatności.

Listing 8. Dockerfile dla nowego obrazu node:13.10

```
FROM node:13.10

WORKDIR /tmp/python

RUN wget \
https://www.python.org/ftp/python/3.7.3/Python-3.7.3.tar.xz \
&& tar xf Python-3.7.3.tar.xz \
&& sh ./Python-3.7.3/configure \
&& make && make install \
&& update-alternatives --install \
/usr/bin/python python /usr/local/bin/python3.7 10 \
&& ln -s /usr/share/pyshared/lsb_release.py \
/usr/local/lib/python3.7/site-packages/lsb_release.py \
&& pip3 install --upgrade pip \
&& rm -rf /tmp/python
```

Celem wszystkich poleceń występujących w instrukcji `RUN` jest zainstalowanie nowszej i wolnej od podatności wersji Pythona. W Listingu 9 przedstawiono przykładowe polecenie, które może posłużyć do zbudowania własnego obrazu NodeJS.

Listing 9. Budowanie nowego obrazu node:13.10

```
$ docker image build -t secure-node:13.10 .
```

Nowo zbudowany obraz `secure-node:13.10` może zostać wykorzystany jako obraz bazowy w pliku Dockerfile. Przykład końcowego Dockerfile'a został przedstawiony w Listingu 10.

Listing 10. Końcowy plik Dockerfile

```
FROM secure-node:13.10

WORKDIR /usr/src/app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
CMD ["node", "server.js"]
```

Usuwanie podatności – sposób drugi

Drugim sposobem na usunięcie podatności z obrazu jest uruchomienie kontenera w trybie interaktywnym na podstawie obrazu zawierającego podatności. Aby to zrobić, należy użyć polecenia przedstawionego w Listingu 11.

Listing 11. Uruchomienie kontenera w trybie interaktywnym

```
$ docker container run -it --name my-node node:13.10 /bin/bash
```

Powyższe polecenie spowoduje automatyczne uruchomienie kontenera i przekierowanie strumieni standardowych do powłoki. Daje nam to możliwość zainstalowania nowszej wersji dla bibliotek i paczek zawierających podatności. Jako przykład usuwania podatności, w Listingu 12 zostały przedstawione polecenia służące do zainstalowania nowszej wersji Pythona.

Listing 12. Przykład usuwanie podatności wewnątrz działającego kontenera

```
$ mkdir /tmp/python && cd /tmp/python
$ wget \
https://www.python.org/ftp/python/3.7.3/Python-3.7.3.tar.xz \
&& tar xf Python-3.7.3.tar.xz \
&& sh ./Python-3.7.3/configure \
&& make && make install \
&& update-alternatives --install \
/usr/bin/python python /usr/local/bin/python3.7 10 \
&& ln -s /usr/share/pyshared/lsb_release.py \
/usr/local/lib/python3.7/site-packages/lsb_release.py \
&& pip3 install --upgrade pip \
&& rm -rf /tmp/python
$ exit
```

Ostatni krok to utworzenie obrazu na podstawie wcześniej utworzonego kontenera, w którym usunięte zostały podatności. W tym celu należy skorzystać z polecenia przedstawionego w Listingu 13.

Listing 13. Utworzenie obrazu na podstawie kontenera

```
$ docker container commit my-node secure-node:13.10
```

Za pomocą powyższego polecenia, na podstawie kontenera o nazwie `my-node`, utworzono obraz `secure-node:13.10`, który może być wykorzystany jako obraz bazowy dla aplikacji NodeJS.

Podsumowanie

Podsumowanie będzie bardzo krótkie, ale mam nadzieję dla Ciebie treściwe (nie lubię wodolejstwa). Musisz zapamiętać, że:

- obrazy dockerowe dziedziczą warstwy z obrazów bazowych, co może wpływać na bezpieczeństwo obrazu końcowego,
- narzędzia do skanowania obrazów, takie jak Trivy, Gype i Clair, pomagają w identyfikacji i ocenie ryzyka podatności w obrazach,
- nawet oficjalne obrazy dockerowe (dostępne na Docker Hub) nie gwarantują bezpieczeństwa (polecam tutaj podejście "Ufam, ale sprawdzam")
- aby usunąć podatności z obrazów można utworzyć plik Dockerfile, w którym dodawane są łatki dla podatności, lub można uruchomić kontener w trybie interaktywnym i zaktualizować biblioteki zawierające podatności, a następnie zapisać go do nowego obrazu.

W tym e-booku to wszystko. Oczywiście temat bezpieczeństwa aplikacji jest znacznie szerszy (co nazywane jest *łańcuchem dostaw oprogramowania* – ang. Software Supply Chain) i przyjrzymy się temu znacznie głębiej w szkoleniu [Automation Maestro](#).



Damian Naprawa

CEO akademia.wkontenerach.pl

email: akademia@wkontenerach.pl



wkontenerach.pl

 **Zobacz ofertę szkoleń:** <https://akademia.wkontenerach.pl/>



WKONTENERACH.PL - AKADEMIA